

Keyspace Operations

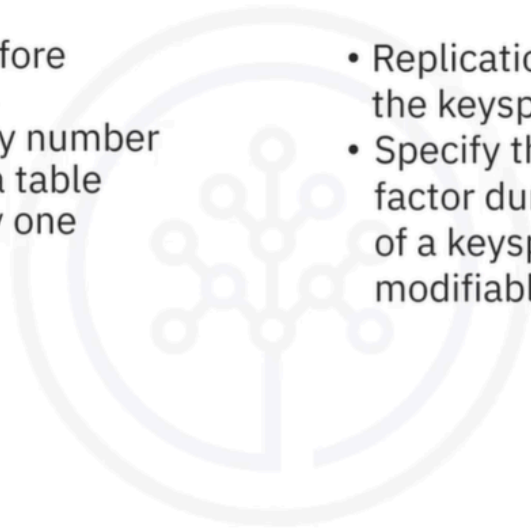
Sure! Let's talk about keyspaces in Apache Cassandra, which is a NoSQL database.

A keyspace is like a container or a folder where you store your data tables. Before you can create any tables, you need to define a keyspace because there isn't a default one. Think of it as setting up a new filing cabinet before you start filing away your documents. Each keyspace can hold many tables, but each table belongs to only one keyspace.

Now, when you create a keyspace, you also decide how many copies of your data you want to keep (this is called the replication factor) and how those copies are distributed across different servers (this is known as the replication strategy). For example, if you have a keyspace that replicates data three times, it means there are three copies of your data stored on different servers to ensure that if one server fails, your data is still safe and accessible from another server.

Keyspaces

- Must define before creating tables
- Can contain any number of tables, and a table belongs to only one keyspace
- Replication is specified at the keyspace level
- Specify the replication factor during the creation of a keyspace and is modifiable



- Let's go into more details regarding keyspaces and keyspace operations. A keyspace needs to be defined before creating tables, as there is no default keyspace. A keyspace can contain any number of tables and a table belongs to only one keyspace. Replication is specified at the keyspace level, and you specify the replication factor during the creation of a keyspace but the replication factor can be modified later.

Create a keyspace



- Let's take a CQL, CREATE KEYSPACE example, you can see that besides the name, we need to set two parameters, class, which refers to the replication strategy, and the replication factor, which is set at each datacenter level. In this example, we are creating a keyspace called `intro_cassandra` that replicates its data, AKA tables partitions five times between the cluster nodes, in 3 nodes from `datacenter1` and 2 nodes from `datacenter2`. To check if the keyspace has been created, you can use `DESCRIBE KEYSPACES` or `DESCRIBE` then the name of the keyspace.

Data replication

- Replication Factor
 - Number of replicas placed on different nodes in the cluster
 - Replication Strategy
 - Which nodes are going to house the replicas
 - Replicas
 - All replicas are equally important with no primary or secondary replicas
 - Replication factor should not exceed the number of cluster nodes
- Replication factor refers to the number of replicas of data placed on different nodes while replication strategy determines on which cluster nodes the replicas are going to be located. After data is initially distributed according to partition key hashed and token pre-allocation, then data is also replicated according to these two pieces of information, replication factor and replication strategy. Before we move on to some examples of replication factor and strategy, two important notes regarding the replicas. In Apache Cassandra, all replicas are equally important. There are no primary or secondary replicas and as a general rule, the replication factor should not exceed the number of nodes in the cluster.

Single DC cluster: Rep factor 3

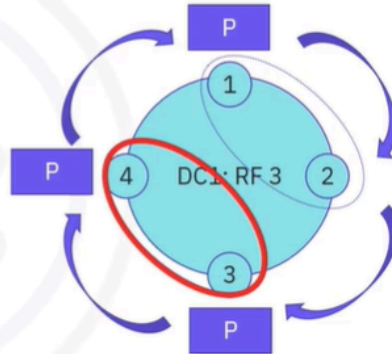
```
CREATE KEYSPACE intro_cassandra
```

```
WITH REPLICATION = { 'class' :
```

```
'NetworkTopologyStrategy',
```

```
'datacenter1' : 3 };
```

//Node 1 and 2 are in the same rack so that the second replica is placed in node 3



Single DC cluster: Rep factor 3

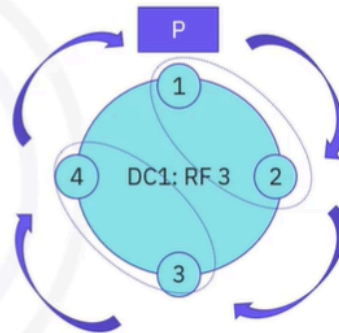
```
CREATE KEYSPACE intro_cassandra
```

```
WITH REPLICATION = { 'class' :
```

```
'NetworkTopologyStrategy',
```

```
'datacenter1' : 3 };
```

//Node 1 and 2 are in the same rack so that the second replica is placed in node 3



Single DC cluster: Rep factor 3

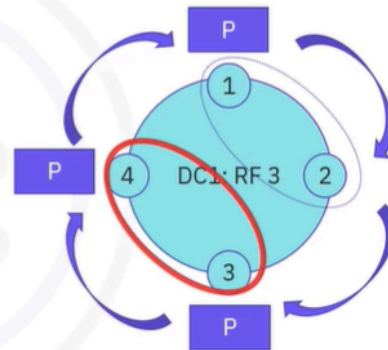
```
CREATE KEYSPACE intro_cassandra
```

```
WITH REPLICATION = { 'class' :
```

```
'NetworkTopologyStrategy',
```

```
'datacenter1' : 3 };
```

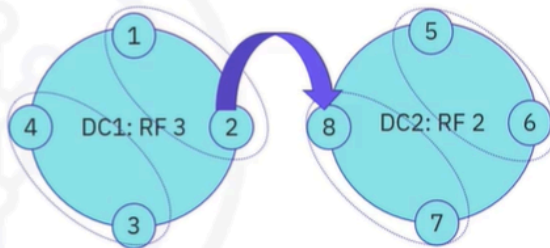
```
//Node 1 and 2 are in the same rack so that the  
second replica is placed in node 3
```



- Let's take a simple example of a four-node Cassandra cluster with one datacenter and a keyspace called intro_cassandra defined with replication factor 3. In CQL, when we create a keyspace, we need to specify the class as being network topology strategy, the only option recommended for production systems, and specify the replication at datacenter level, in this case, 3.
- Cluster topology indicates that nodes 1 and 2 are in the same rack, while 3 and 4 share another rack as well, we assume that our partition, named P in the diagram has been initially allocated to node 1. We have a replication factor of 3, meaning we need to replicate data into two more nodes in order to reach the replication factor of 3. Data replication is done clockwise in the cluster while taking into consideration the rack allocation of the servers.
- Since nodes 1 and 2 are in the same rack, Cassandra will try to place the next replica on a node from a different rack, in this case, node 3 and the last replica will be placed on node 4. Since two replicas are placed in different racks, the fact that nodes 3 and 4 are in the same rack will not be an issue.

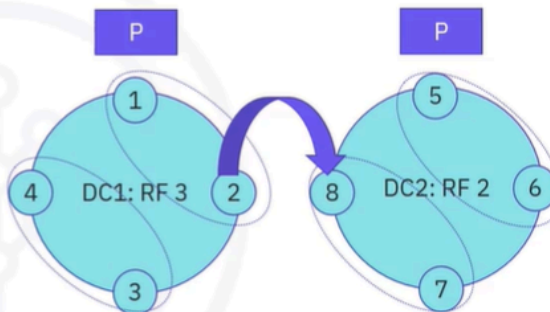
Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



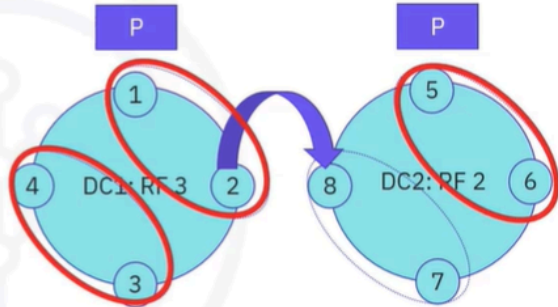
Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



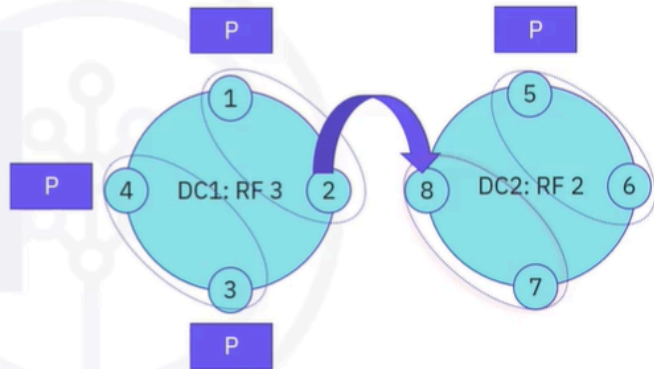
Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



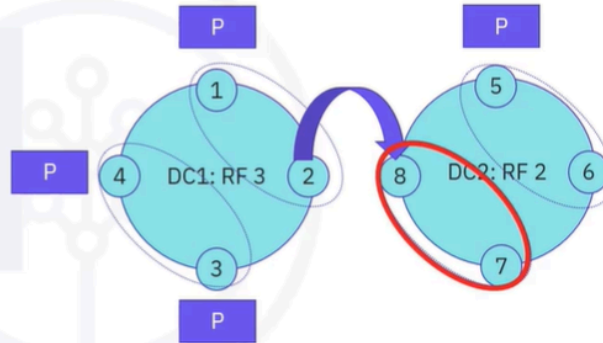
Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



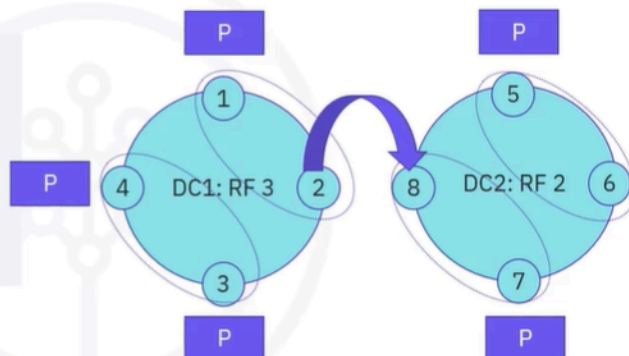
Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



Multiple DC cluster: Rep factor 5

```
CREATE KEYSPACE intro_cassandra WITH  
REPLICATION = { 'class' :  
'NetworkTopologyStrategy',  
'datacenter1' : 3, 'datacenter2' : 2 };
```



- Let's now take an example in which the cluster is deployed in a multidata center environment. Here we have two data centers, and our keyspace has a replication factor of 5.
- Just like in the previous example, we need to use CQL to create a keyspace, and again, we specify the class as being network topology strategy and we specify the replication factor at the datacenter level as being 3 for datacenter1 and 2 for datacenter2.

- We assume that our partition, named P in the diagram, has been initially allocated to node 1 in datacenter1 and node 5 in datacenter2. Just like in the previous example, in datacenter1, two more replicas are going to be placed clockwise according to the rack placements of the servers thus nodes 3 and 4 will get the replicas of partition P.
- In our case, we have two datacenters, so after completing the first datacenter replica placement, Cassandra will go to the next datacenter in order to place the other two replicas.
- Since the partition P has already been allocated initially to node 5 in datacenter2, we need to now place only one more replica in datacenter2.

Alter a keyspace

```
ALTER KEYSPACE intro_cassandra  
  WITH REPLICATION = {  
    'class' : 'NetworkTopologyStrategy',  
    'datacenter1' : 3,  
    'datacenter2' : 3};
```

Modifies the keyspace replication factor

Replication factor: The number of copies of the data Cassandra creates in each data center

- Taking into consideration the rack placement of the nodes, node 7 is chosen to hold the fifth replica of our data, and this is how our five replicas ended up in the Cassandra cluster nodes. We have already seen in this video examples of using CREATE KEYSPACE, but now let's see how we can modify or delete a keyspace. You can modify an existing keyspace, for example, modify the replication factor of a keyspace by using the CQL, ALTER KEYSPACE command. An example, you can increase the overall keyspace replication factor from 5 to 6 by increasing the replication factor in datacenter2 from 2 to 3.

Drop a keyspace

```
DROP KEYSPACE intro_cassandra;
```

- Immediate removal of the keyspace, including objects such as tables, functions, and data the keyspace contains
- Cassandra takes a snapshot before dropping the keyspace

- You can also delete or drop a keyspace by using the DROP KEYSPACE command in CQL, which will lead to the removal of all the keyspace tables and the data those contain. To protect the system against an unwanted drop, Cassandra will by default take a snapshot of the keyspace prior to executing the drop. This means that you will be able to recover your data in case you need to.

Recap

In this video, you have learned that:

- Keyspaces are defined before creating tables, and a keyspace can contain any number of tables
- Replication is specified at the keyspace level
- Data replication depends on a replication factor and a replication strategy, both set at the keyspace level
- Replication factor sets the number of replicas
- Replication strategy determines on which cluster nodes the replicas are going to be located
- Common keyspace operations are CREATE KEYSPACE, ALTER KEYSPACE, and DROP KEYSPACE